

Web Security Fundamentals

1. Overview

Web security fundamentals are the basic ideas used to protect applications from common attacks such as SQL injection, XSS, CSRF, password cracking, brute force abuse, invalid input, and insecure network communication.

Definition

Security in backend and web systems means protecting:

- Data from unauthorized access
- Users from account takeover
- Servers from malicious requests
- Applications from broken logic and unsafe input
- Communication from interception or tampering

This guide focuses on seven placement-interview topics:

Topic	Main Problem It Solves
SQL Injection	Attacker manipulates database queries
XSS	Attacker runs JavaScript in another user's browser
CSRF	Attacker tricks a logged-in user into performing an unwanted action
Password Hashing	Protects passwords even if the database leaks
Rate Limiting	Controls request abuse and brute force attacks
Input Validation	Rejects malformed, unsafe, or unexpected input
HTTPS	Encrypts and authenticates client-server communication

Why It Matters

Security matters because real systems store sensitive information:

- Login credentials
- Bank details
- Personal data
- Medical records
- Private messages
- Company secrets
- Payment information

A small mistake such as string-concatenating SQL or storing plain-text passwords can lead to a serious data breach.

Where It Is Used in Real Systems

These concepts appear in:

- Backend APIs
- Login and signup systems
- Payment systems
- Admin dashboards
- Browsers
- Databases
- Authentication systems
- Cloud services
- Online assessment backend questions
- SDE placement interview discussions

Why Interviewers Ask About It

Interviewers ask security fundamentals because they test whether you can build production-safe software, not just working software.

They want to know if you understand:

- How attackers think
- Why trusting user input is dangerous
- How authentication and sessions can be abused
- How to protect data at rest and in transit
- How to explain practical prevention techniques clearly

2. Core Idea

The core idea of web security is:

Never blindly trust input, identity, storage, or network communication.

Every request, parameter, cookie, form field, file upload, and HTTP header can be controlled or influenced by an attacker.

Intuition

Think of a backend server like a bank counter.

A customer may say:

"Transfer money from my account to this account."

The bank should not immediately obey. It must check:

- Is this person authenticated?
- Is this person authorized?
- Is the request valid?
- Is the amount allowed?

- Is this request suspiciously repeated?
- Is the communication secure?
- Is the database operation safe?

Web security applies the same discipline to software.

Real-World Analogy

Real World	Web Security Equivalent
ID card check	Authentication
Permission check	Authorization
Form verification	Input validation
Locking confidential files	Password hashing and encryption
CCTV and request limits	Rate limiting and monitoring
Tamper-proof sealed envelope	HTTPS
Preventing fake instructions	CSRF protection

Small Example

Unsafe login query:

```
SELECT * FROM users
WHERE email = 'input_email'
AND password = 'input_password';
```

If the application builds this query by string concatenation, an attacker may enter:

```
' OR '1'='1
```

The query can become logically true and may bypass login.

Safe approach:

```
SELECT * FROM users
WHERE email = ?
AND password_hash = ?;
```

Here, user input is treated as data, not executable SQL.

Step-by-Step Explanation

1. A browser or client sends a request.
2. The server receives input from URL parameters, body, headers, cookies, or files.
3. The server validates and sanitizes input where needed.
4. The server checks authentication and authorization.
5. The server safely interacts with databases and external services.
6. The server sends output with proper encoding and security headers.
7. HTTPS protects the request and response while they travel over the network.

3. Important Subtopics

3.1 SQL Injection

What It Means

SQL injection happens when attacker-controlled input becomes part of an SQL query's executable logic.

Unsafe example:

```
query = "SELECT * FROM users WHERE email = '" + email + "'"
```

If `email` contains SQL syntax, the database may execute attacker-controlled logic.

Why It Matters

SQL injection can allow attackers to:

- Bypass login
- Read private data
- Modify records
- Delete tables
- Dump the full database
- Escalate privileges

Example

Input:

```
' OR '1'='1
```

Unsafe query:

```
SELECT * FROM users  
WHERE email = '' OR '1'='1';
```

Because `'1'='1'` is always true, the query may return unintended rows.

Common Interview Angle

Interviewers usually ask:

- What is SQL injection?
- How does it happen?
- How do prepared statements prevent it?
- Is input validation alone enough?

Expected answer: use parameterized queries/prepared statements, least privilege database users, ORM safety, input validation, and careful error handling.

3.2 Cross-Site Scripting (XSS)

What It Means

XSS occurs when attacker-controlled JavaScript runs in another user's browser because the application outputs untrusted content without proper encoding.

Why It Matters

XSS can allow attackers to:

- Steal session tokens if accessible
- Perform actions as the victim
- Modify page content
- Redirect users to phishing pages
- Capture keystrokes

Example

Suppose a comment system stores this input:

```
<script>alert("hacked")</script>
```

If the website renders it directly, the script executes in every visitor's browser.

Safe output should escape it:

```
&lt;script&gt;alert("hacked")&lt;/script&gt;
```

Common Interview Angle

Interviewers expect you to mention:

- Output encoding
- Sanitization for allowed HTML

- Content Security Policy
- **HttpOnly** cookies
- Difference between stored, reflected, and DOM-based XSS

3.3 Cross-Site Request Forgery (CSRF)

What It Means

CSRF tricks a logged-in user's browser into sending an unwanted request to a trusted website.

The attacker does not need to know the user's password. The browser automatically attaches cookies for the trusted site.

Why It Matters

CSRF can perform unwanted actions such as:

- Changing email
- Changing password
- Making a payment
- Updating profile data
- Submitting an admin action

Example

If a banking site supports:

```
POST /transfer  
amount=10000&to=attacker
```

An attacker may place a hidden form on another site. If the victim is logged into the bank, the browser may send the bank cookies automatically.

Common Interview Angle

Interviewers expect:

- CSRF token explanation
- SameSite cookies
- Checking origin/referer headers
- Using unsafe methods only for state-changing operations
- Difference between XSS and CSRF

3.4 Password Hashing

What It Means

Password hashing means storing a one-way hash of the password instead of the actual password.

During login:

1. User enters password.
2. Server hashes the entered password using the same password hashing algorithm.
3. Server compares the result with the stored hash.

Why It Matters

If a database leaks, plain-text passwords expose every user immediately. Hashing makes password recovery much harder.

Example

Bad storage:

email	password
user@example.com	myPassword123

Better storage:

email	password_hash
user@example.com	bcrypt_hash_value

Common Interview Angle

Interviewers expect you to mention:

- Never store plain-text passwords
- Use slow password hashing algorithms such as bcrypt, scrypt, or Argon2
- Use a unique salt per password
- Hashing is different from encryption
- Do not use fast general hashes like MD5 or SHA-256 directly for passwords

3.5 Rate Limiting

What It Means

Rate limiting restricts how many requests a user, IP address, token, or client can make in a given time period.

Why It Matters

Rate limiting helps prevent:

- Brute force login attempts
- OTP guessing
- API abuse
- Scraping
- Denial-of-service amplification

- Expensive endpoint overuse

Example

Login endpoint rule:

```
Allow only 5 failed login attempts per account per 15 minutes.
```

API rule:

```
Allow 100 requests per API key per minute.
```

Common Interview Angle

Interviewers may ask:

- How would you design rate limiting?
- What key would you limit on?
- Difference between fixed window, sliding window, token bucket, and leaky bucket?
- How do you rate limit in a distributed system?

3.6 Input Validation

What It Means

Input validation means checking whether incoming data has the expected type, format, length, range, and business meaning before using it.

Why It Matters

Invalid input can cause:

- Security bugs
- Crashes
- Data corruption
- Broken business rules
- Injection attacks
- Unexpected behavior

Example

For a signup form:

Field	Validation
Email	Valid email format, max length

Field	Validation
Password	Minimum length and complexity rules
Age	Integer within valid range
Phone	Digits and country-specific format
Role	Must be from allowed values

Common Interview Angle

Interviewers expect:

- Validate on server side, not only client side
- Prefer allowlists over blocklists
- Check length, type, range, and format
- Validation is not the same as output encoding
- Validation reduces risk but does not replace prepared statements or authorization checks

3.7 HTTPS

What It Means

HTTPS is HTTP over TLS. It protects communication between client and server using encryption, authentication, and integrity.

Why It Matters

Without HTTPS, attackers on the network can:

- Read passwords
- Steal cookies
- Modify responses
- Inject malicious scripts
- Redirect users

Example

With plain HTTP:

```
User -> Wi-Fi -> Server  
password=secret123 is visible to network attackers
```

With HTTPS:

User -> encrypted TLS tunnel -> Server
network attackers cannot read or silently modify traffic

Common Interview Angle

Interviewers expect:

- HTTPS provides encryption, integrity, and server authentication
- Certificates prove server identity
- TLS handshake establishes shared keys
- HTTPS does not automatically fix application-level bugs

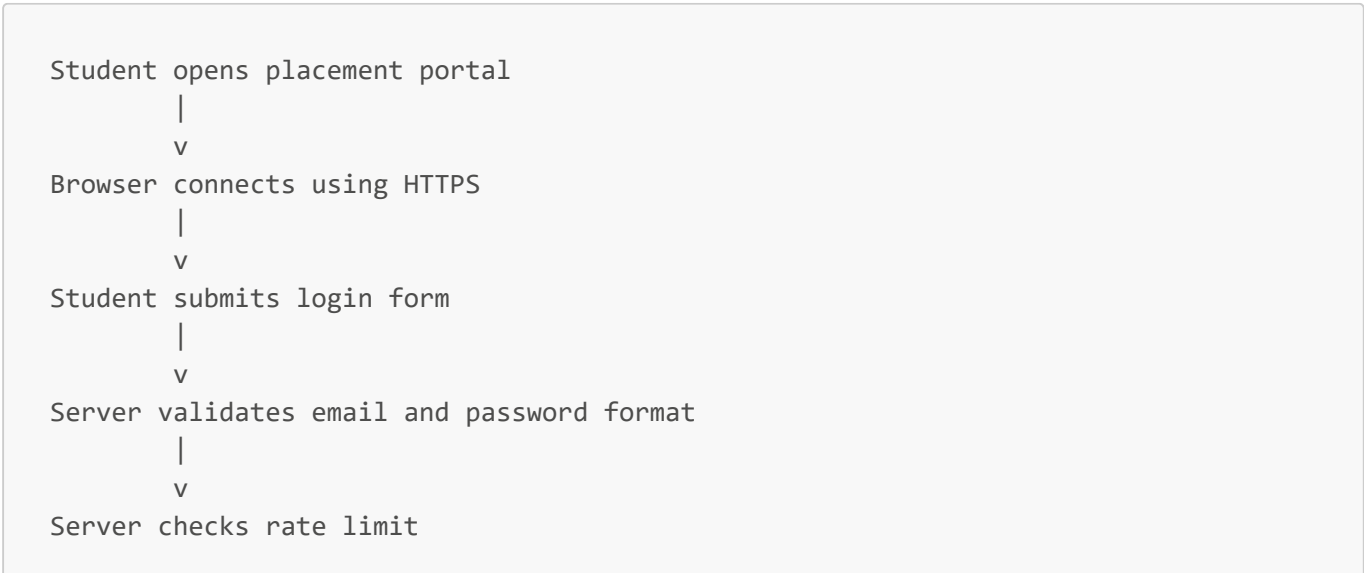
4. Real-World Example

Consider a placement portal where students log in, upload resumes, apply to companies, and view interview schedules.

Security Use Cases

Feature	Security Concern	Protection
Login	Password guessing	Password hashing and rate limiting
Student search	SQL injection	Parameterized queries
Resume upload	Malicious files	File validation and storage controls
Profile bio	XSS	Output encoding and sanitization
Apply button	CSRF	CSRF token or SameSite cookies
Admin panel	Unauthorized changes	Authentication and authorization
API calls	Traffic interception	HTTPS

Example Flow



```
    |
    v
Server compares password using bcrypt/Argon2
    |
    v
Server creates secure session cookie
    |
    v
Student submits application form
    |
    v
Server checks CSRF token, validates input, uses safe SQL query
```

5. Diagrams / Mental Models

Request Security Pipeline

```
Client Request
  |
  v
HTTPS protects traffic
  |
  v
Authentication: Who are you?
  |
  v
Authorization: Are you allowed?
  |
  v
Rate limiting: Are you abusing the system?
  |
  v
Input validation: Is the data acceptable?
  |
  v
Safe processing: Prepared SQL, safe file handling
  |
  v
Safe output: Encoding, security headers
  |
  v
Response
```

SQL Injection Mental Model

```
Unsafe:
User input + SQL string = executable SQL
```

Safe:
SQL template + parameters = input treated as data

XSS vs CSRF Mental Model

Attack	Attacker Abuses	Victim Impact
XSS	Trust in website content	Malicious script runs in browser
CSRF	Trust in user's browser cookies	Unwanted authenticated request sent

Password Storage Flow

Signup:
password -> salt + slow hash -> store hash

Login:
entered password -> same hash check -> compare with stored hash

Rate Limiting Flow

```
Request arrives
  |
  v
Identify key: IP/user/API key/account
  |
  v
Check request count in time window
  |
  +--> Within limit: allow
  |
  +--> Over limit: reject or slow down
```

6. Common Interview Questions

1. What is SQL injection?

SQL injection is a vulnerability where attacker-controlled input becomes part of an SQL query's executable logic.

Key points interviewer expects:

- Happens due to unsafe query construction
- Can bypass login or leak data
- Prevented using parameterized queries/prepared statements

Common mistakes:

- Saying input validation alone fully prevents it
- Only giving the ' OR '1'='1 example without explaining root cause
- Forgetting database least privilege

2. How do prepared statements prevent SQL injection?

Prepared statements separate SQL code from user data. The SQL structure is compiled first, and user values are passed later as parameters.

Key points interviewer expects:

- Query and data are separated
- User input is not interpreted as SQL syntax
- Works better than manual escaping

Common mistakes:

- Saying prepared statements "remove bad characters"
- Confusing prepared statements with string concatenation

3. What is XSS?

XSS is a vulnerability where malicious JavaScript runs in a victim's browser because untrusted content is rendered unsafely.

Key points interviewer expects:

- Browser executes attacker-controlled script
- Can steal data or perform actions
- Prevent with output encoding, sanitization, CSP, and secure cookies

Common mistakes:

- Saying XSS attacks the server directly
- Ignoring stored vs reflected vs DOM-based XSS

4. What is the difference between stored XSS and reflected XSS?

Stored XSS is saved on the server, such as in a comment or profile field, and affects future users. Reflected XSS is immediately returned in the response, often through a URL or search parameter.

Key points interviewer expects:

- Stored XSS persists
- Reflected XSS usually needs a crafted link
- Both require unsafe output handling

Common mistakes:

- Saying reflected XSS is harmless
- Thinking only `<script>` tags can cause XSS

5. What is CSRF?

CSRF is an attack where a malicious site tricks a logged-in user's browser into sending an unwanted request to another trusted site.

Key points interviewer expects:

- Uses browser's automatic cookie sending
- Targets state-changing actions
- Prevented with CSRF tokens and SameSite cookies

Common mistakes:

- Confusing CSRF with XSS
- Saying HTTPS prevents CSRF

6. How do CSRF tokens work?

A CSRF token is a secret random value included in legitimate forms or requests. The server verifies that the token in the request matches the expected token.

Key points interviewer expects:

- Attacker cannot easily know the token
- Token is checked on state-changing requests
- Token should be unpredictable

Common mistakes:

- Using a fixed token for all users
- Checking tokens only on login

7. Why should passwords be hashed instead of encrypted?

Passwords should be hashed because the server does not need to recover the original password. It only needs to verify whether the entered password matches.

Key points interviewer expects:

- Hashing is one-way
- Encryption is reversible with a key
- Use slow password hashing algorithms

Common mistakes:

- Saying hashing and encryption are the same
- Suggesting MD5 or SHA-256 directly for password storage

8. What is a salt in password hashing?

A salt is a unique random value added to each password before hashing.

Key points interviewer expects:

- Prevents identical passwords from having identical hashes
- Makes precomputed rainbow table attacks harder
- Salt does not need to be secret

Common mistakes:

- Saying salt is a secret key
- Reusing the same salt for every user

9. What is rate limiting?

Rate limiting restricts how many requests a client can make in a time period.

Key points interviewer expects:

- Prevents abuse and brute force attempts
- Can be based on IP, user ID, account, token, or endpoint
- Common algorithms include fixed window, sliding window, token bucket, and leaky bucket

Common mistakes:

- Rate limiting only by IP for login security
- Ignoring distributed systems and shared storage

10. Why is server-side input validation necessary?

Client-side validation can be bypassed because attackers can send requests directly using tools like curl, Postman, or custom scripts.

Key points interviewer expects:

- Always validate on server
- Client validation improves user experience only
- Validate type, length, format, range, and allowed values

Common mistakes:

- Trusting frontend validation
- Using only blocklists

11. What does HTTPS protect against?

HTTPS protects data in transit by providing encryption, integrity, and server authentication.

Key points interviewer expects:

- Prevents network attackers from reading or modifying traffic
- Uses TLS certificates
- Does not prevent SQL injection, XSS, or broken access control

Common mistakes:

- Saying HTTPS makes a site fully secure

- Ignoring certificate validation

12. What is the difference between authentication and authorization?

Authentication verifies identity. Authorization checks permissions.

Example:

- Authentication: "Are you logged in as Ravi?"
- Authorization: "Is Ravi allowed to access admin reports?"

Key points interviewer expects:

- AuthN means identity
- AuthZ means permission
- Both are needed

Common mistakes:

- Using the two terms interchangeably
- Checking login but not resource ownership

7. Deep-Dive Questions

1. If you use an ORM, are you fully protected from SQL injection?

Not always. ORMs often use parameterized queries by default, but SQL injection can still happen if developers use raw queries, unsafe string concatenation, dynamic table names, or unsafe order/filter clauses.

Best answer:

- ORM reduces risk
- Prepared statements are still the key idea
- Raw SQL must be handled carefully
- Dynamic identifiers need allowlists

2. How would you prevent XSS in a rich text editor that allows some HTML?

Use a trusted HTML sanitizer that allows only safe tags and attributes. Also use output encoding, Content Security Policy, and avoid allowing dangerous attributes such as inline event handlers.

Important points:

- Escaping everything would break rich text
- Sanitization is needed when some HTML is allowed
- Allowlist safe tags like `b`, `i`, `p`, `ul`, `li`
- Block scripts, `javascript:` URLs, and event attributes like `onclick`

3. How do you design rate limiting for login attempts?

Use multiple keys:

- Per account

- Per IP
- Per IP plus account pair
- Per device or session when available

Add progressive delays, temporary lockouts, alerts for suspicious attempts, and avoid revealing whether an email exists.

Do not rely only on IP because attackers can rotate IPs. Do not permanently lock accounts too easily because that creates a denial-of-service risk.

4. Why are bcrypt, scrypt, and Argon2 preferred for passwords?

They are intentionally slow and designed for password hashing. Argon2 and scrypt can also be memory-hard, making large-scale cracking more expensive.

Important points:

- Password attackers try billions of guesses
- Fast hashes help attackers
- Slow and configurable algorithms increase cracking cost
- Each password should have a unique salt

5. Can HTTPS prevent man-in-the-middle attacks completely?

HTTPS prevents typical network-level man-in-the-middle attacks only if certificate validation is correct and the client trusts the right certificate authority.

It can fail if:

- User ignores certificate warnings
- Certificate authority is compromised
- Device has malicious root certificates installed
- Application disables certificate verification

HTTPS protects transport, not insecure backend logic.

8. Comparison Tables

SQL Injection vs XSS vs CSRF

Feature	SQL Injection	XSS	CSRF
Target	Database	Browser/user	Logged-in session
Root Cause	Unsafe SQL construction	Unsafe output rendering	Trusting cookie-based requests
Attacker Goal	Read/modify data	Run script in victim browser	Perform unwanted action
Main Defense	Prepared statements	Output encoding/sanitization	CSRF tokens, SameSite cookies
Example	' OR '1'='1	<script>...</script>	Hidden form submits transfer

Hashing vs Encryption

Feature	Hashing	Encryption
Direction	One-way	Reversible
Needs Key?	Not usually for basic hashing	Yes
Used For Passwords?	Yes, with password hashing algorithms	Usually no
Output Check	Compare hashes	Decrypt ciphertext
Example	bcrypt password hash	AES encrypted message

Authentication vs Authorization

Feature	Authentication	Authorization
Question	Who are you?	What can you access?
Example	Login with password	Admin can delete user
Failure Code Often Used	401 Unauthorized	403 Forbidden
Data Needed	Identity proof	Permissions/roles/ownership

Validation vs Sanitization vs Encoding

Concept	Meaning	Example
Validation	Check input is acceptable	Age must be 18 to 100
Sanitization	Remove or clean unsafe parts	Strip unsafe HTML tags
Encoding	Make output safe for context	Convert <code><</code> to <code>&lt;</code>

Rate Limiting Algorithms

Algorithm	Idea	Pros	Cons
Fixed Window	Count requests per fixed interval	Simple	Allows bursts at boundaries
Sliding Window	Tracks recent rolling interval	More accurate	More storage/complexity
Token Bucket	Tokens refill over time	Allows controlled bursts	Needs careful tuning
Leaky Bucket	Processes at steady rate	Smooth output	May delay or drop bursts

HTTP vs HTTPS

Feature	HTTP	HTTPS
Encryption	No	Yes
Integrity	No strong protection	Protected by TLS
Server Authentication	No	Certificate-based
Safe on Public Wi-Fi	No	Much safer
Prevents App Bugs	No	No

9. Common Mistakes

- Thinking HTTPS makes the whole application secure.
- Storing passwords in plain text.
- Using MD5 or SHA-256 directly for password storage.
- Trusting client-side validation.
- Building SQL queries using string concatenation.
- Assuming ORMs automatically prevent every SQL injection case.
- Confusing XSS and CSRF.
- Forgetting output encoding while focusing only on input validation.
- Using one global salt for all passwords.
- Rate limiting only by IP address.
- Returning detailed error messages like "email exists but password is wrong."
- Putting sensitive tokens in URLs.
- Forgetting authorization after authentication.
- Allowing state-changing actions through GET requests.
- Ignoring security headers such as CSP, HSTS, and cookie flags.

10. Edge Cases / Special Cases

SQL Injection Edge Cases

- Dynamic **ORDER BY** fields cannot usually be parameterized as values; use allowlists.
- Table and column names should not come directly from user input.
- Error messages can leak database structure.
- Second-order SQL injection happens when malicious input is stored first and executed later.

XSS Edge Cases

- XSS can happen without **<script>** tags, such as through event handlers or unsafe URLs.
- DOM-based XSS may occur entirely in frontend JavaScript.
- Output encoding must match context: HTML body, attribute, JavaScript, URL, and CSS contexts differ.
- **HttpOnly** cookies reduce cookie theft but do not stop all XSS impact.

CSRF Edge Cases

- CSRF mainly affects cookie-based authentication.

- APIs using bearer tokens in **Authorization** headers are less naturally exposed to CSRF if attackers cannot set that header cross-site.
- SameSite cookies help, but token-based protection may still be needed for high-risk actions.
- GET requests should not change server state.

Password Hashing Edge Cases

- Password reset tokens should be random, time-limited, and stored safely.
- Password comparison should avoid timing leaks where relevant.
- Hash cost factors must be tuned so login is not too slow but cracking remains expensive.
- Passwords should be checked against breached-password lists when possible.

Rate Limiting Edge Cases

- Distributed systems need shared counters, often using Redis or a similar store.
- NAT can make many real users share one IP.
- Attackers may rotate IPs.
- Locking accounts too aggressively can let attackers block real users.

HTTPS Edge Cases

- Mixed content can load insecure resources on an HTTPS page.
- Expired or misconfigured certificates break trust.
- HSTS tells browsers to use HTTPS automatically for future visits.
- HTTPS protects data in transit, not data at rest.

11. How to Explain in Interview

Security means designing the application so that untrusted input, stolen traffic, leaked databases, and malicious requests do not easily compromise users or data. For web applications, I focus on prepared statements to stop SQL injection, output encoding and sanitization to prevent XSS, CSRF tokens or SameSite cookies for unwanted authenticated requests, slow salted password hashing for stored credentials, rate limiting for abuse control, server-side validation for all inputs, and HTTPS for encrypted and authenticated communication.

12. Quick Revision Notes

Key Definitions

Term	Meaning
SQL Injection	Attacker changes SQL logic through input
XSS	Attacker runs JavaScript in victim's browser
CSRF	Attacker causes victim's browser to send unwanted authenticated request
Password Hashing	One-way storage of password verifier
Salt	Unique random value added before password hashing

Term	Meaning
Rate Limiting	Restrict requests per time period
Input Validation	Check input type, format, length, and range
HTTPS	HTTP over TLS for secure transport

Important Points

- Use prepared statements for SQL queries.
- Encode output to prevent XSS.
- Use CSRF tokens for state-changing cookie-authenticated requests.
- Store passwords using bcrypt, scrypt, or Argon2.
- Validate input on the server.
- Rate limit login and expensive APIs.
- Use HTTPS everywhere.
- Authentication is identity; authorization is permission.

Common Comparisons

Comparison	One-Line Difference
Hashing vs Encryption	Hashing is one-way; encryption is reversible
XSS vs CSRF	XSS runs script; CSRF sends unwanted request
Validation vs Encoding	Validation checks input; encoding makes output safe
HTTP vs HTTPS	HTTPS adds TLS security
Authentication vs Authorization	Identity vs permission

Must-Remember Facts

- Never store plain-text passwords.
- Never build SQL with raw string concatenation.
- Client-side validation is not security.
- HTTPS does not fix application vulnerabilities.
- XSS can defeat many client-side protections.
- CSRF is especially relevant for cookie-based sessions.
- Rate limiting should consider user, IP, account, and endpoint.

Interview Traps

- Saying "sanitize input" as the only answer to SQL injection.
- Recommending MD5 for passwords.
- Saying CSRF steals cookies.
- Saying HTTPS prevents XSS.
- Forgetting authorization after login.
- Treating GET requests as safe for state changes.

13. Practice Tasks

Task 1: Identify SQL Injection Risk

Given:

```
query = "SELECT * FROM users WHERE id = " + user_id
```

Explain why it is unsafe and rewrite it using a parameterized query.

Task 2: XSS Output Encoding

Input:

```
<img src=x onerror=alert(1)>
```

Write what should happen if this input is displayed in:

- A plain text comment
- A rich text comment
- An HTML attribute

Task 3: CSRF Defense Design

Design protection for:

```
POST /change-email
```

Include:

- CSRF token
- SameSite cookie
- Origin check
- Why GET should not be used

Task 4: Password Storage Review

Review this table:

email	password
a@example.com	123456
b@example.com	qwerty

Write a safer schema and explain how login verification should work.

Task 5: Rate Limiter Simulation

Implement a simple fixed-window rate limiter in Python:

```
from time import time

class FixedWindowRateLimiter:
    def __init__(self, limit, window_seconds):
        self.limit = limit
        self.window_seconds = window_seconds
        self.requests = {}

    def allow(self, key):
        now = int(time())
        window = now // self.window_seconds
        bucket = (key, window)

        count = self.requests.get(bucket, 0)
        if count >= self.limit:
            return False

        self.requests[bucket] = count + 1
        return True
```

Then answer:

- What happens at window boundaries?
- Why is sliding window more accurate?
- What changes in a distributed system?

Task 6: HTTPS Explanation

Explain what happens when a browser opens:

```
https://example.com/login
```

Mention:

- DNS lookup
- TCP connection
- TLS handshake
- Certificate validation
- Encrypted HTTP request

Task 7: Security Review of a Login API

Review this login flow:

1. User submits email and password.
2. Server checks database.
3. If wrong password, server returns "password incorrect".
4. Passwords are stored using SHA-256.
5. No rate limit is applied.

Find at least four problems and propose fixes.

14. Final Cheat Sheet

Core Definition

Web security fundamentals protect applications from unsafe input, malicious scripts, forged requests, password leaks, request abuse, and insecure network communication.

Why It Matters

Security protects user data, business logic, money, identity, and trust. In interviews, it shows that you can build production-ready systems, not just code that works for happy paths.

Most Asked Questions

Question	Short Answer
How to prevent SQL injection?	Use prepared statements and avoid SQL string concatenation
How to prevent XSS?	Encode output, sanitize allowed HTML, use CSP
How to prevent CSRF?	Use CSRF tokens, SameSite cookies, and origin checks
How to store passwords?	Use salted bcrypt, scrypt, or Argon2 hashes
Why rate limit?	Prevent brute force, abuse, scraping, and overload
Why server-side validation?	Client-side validation can be bypassed
What does HTTPS provide?	Encryption, integrity, and server authentication

Common Comparisons

A	B	Difference
SQL Injection	XSS	Database attack vs browser script attack
XSS	CSRF	Script execution vs forged authenticated request
Hashing	Encryption	One-way vs reversible
Validation	Sanitization	Check input vs clean input
Authentication	Authorization	Identity vs permission
HTTP	HTTPS	Plain transport vs TLS-protected transport

One-Line Interview Answer

Security in web applications means treating every input and request as untrusted, using prepared SQL queries, safe output encoding, CSRF protection, slow salted password hashing, rate limiting, server-side validation, and HTTPS to protect users and data.